Unité 5 — Cahier de laboratoire (étudiant)

Livrable C — Analyse de binaires & sanitizers · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Environnement isolé (livrable E).

Lab 5.1 — Analyse boîte noire d'un binaire EXERCICE GUIDÉ

1. Objectifs

- Analyser un binaire fourni sans source : chaînes, fonctions, xrefs.
- Localiser une fonction dangereuse et formuler une hypothèse de vulnérabilité.

2. Prérequis

- Cours U5 §2–3.

3. Environnement

Image de référence (livrable E).

4. Fichiers

Binaire `cible_bb` fourni (sans source).

5. Commandes

SHELL

```
strings -n 6 cible_bb | less
objdump -d -M intel cible_bb | grep "<.*>:"
r2 -q -c "aa; afl" cible_bb
r2 -q -c "aa; axt sym.imp.strcpy" cible_bb    # qui appelle strcpy ?
```

6. Manipulations

a. Extrayez les chaînes : devinez le rôle et les entrées du programme.

b. Listez les fonctions (objdump/radare2).

c. Cherchez les références vers `strcpy` / `gets` ; situez l'appel.

d. Formulez une hypothèse : quelle entrée pourrait déborder quoi ?

7. Résultats attendus

Vous identifiez les messages, les fonctions `main` / `func`, et un appel à `strcpy` depuis `main` — piste d'un débordement sur l'argument.

8. Explications

Les chaînes et xrefs suffisent souvent à localiser une entrée non contrôlée menant à une fonction dangereuse, avant toute exécution.

9. Questions

1. Quelles entrées le programme accepte-t-il (indice : chaînes) ?
2. Quelle fonction dangereuse est appelée, et depuis où ?
3. Comment confirmer l'hypothèse dynamiquement (gdb) ?

10. Vulnérabilité & remédiation

Hypothèse : débordement via `strcpy` (CWE-121). Confirmation en Unité 2 (exploitation).

11. Durée

1 h.

12. Corrigé : livrable D, Lab 5.1.

Lab 5.2 — Instrumenter avec ASan et UBSan EXERCICE GUIDÉ

1. Objectifs

- Recompiler du code vulnérable avec ASan et UBSan et lire les rapports.

2. Prérequis

- Lab 5.1 ; Unité 3 (ASan).

3. Environnement

Image de référence (livrable E).

4. Fichiers

INTOV.C (DEPASSEMENT D ENTIER)

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv){
    int taille = (argc>1)? atoi(argv[1]) : 2000000000;
    int total = taille + 1000000000;   /* BUG : depassement d'entier signe possible */
    printf("total = %d\n", total);
    return 0;
}
```

5. Commandes

SHELL

```
clang -g -fsanitize=undefined intov.c -o intov_ubsan && ./intov_ubsan
# programme memoire heap_of.c fourni dans src/
clang -g -fsanitize=address heap_of.c -o hof && ./hof
```

6. Manipulations

a. Compilez `intov.c` avec UBSan ; exécutez : lisez le message d'erreur et la ligne.

b. Reprenez un programme mémoire (Unité 3) avec ASan ; comparez les deux types de diagnostics.

7. Résultats attendus

UBSan signale un `signed integer overflow` avec fichier:ligne ; ASan signale les erreurs mémoire.

8. Explications

UBSan cible les comportements indéfinis arithmétiques/logiques ; ASan cible l'adressage mémoire. Les deux s'activent à la compilation et n'impactent que les builds de test.

9. Questions

1. Quelle ligne UBSan pointe-t-il et pourquoi ?
2. Un dépassement d'entier peut-il mener à une faille mémoire ? (indice : calcul de taille d'allocation)
3. Pourquoi ne pas livrer un binaire ASan/UBSan en production ?

10. Vulnérabilité & remédiation

CWE-190 (integer overflow). Remédiation : vérifier les bornes, types non signés adaptés, additions contrôlées (`__builtin_add_overflow`).

11. Durée

45 min.

12. Corrigé : livrable D, Lab 5.2.

Lab 5.3 — Évaluer la sécurité d'un code source EXERCICE AUTONOME

1. Objectifs

- Mener une évaluation complète (statique + dynamique) d'un code fourni.

2. Prérequis

- Labs 5.1–5.2.

3. Environnement

Image de référence (livrable E).

4. Fichiers

Fichier code.c fourni dans src/ (programme C à auditer).

5. Commandes

SHELL

```
# 1) lecture statique + grep des fonctions dangereuses
grep -nE "gets|strcpy|strcat|sprintf|system|memcpy" code.c
# 2) instrumenter
clang -g -fsanitize=address,undefined code.c -o code_san
# 3) executer : ./code_san <utilisateur> <n>
```

6. Manipulations

a. Repérez statiquement les appels à risque.

b. Instrumentez (ASan+UBSan) et exécutez sur des entrées limites.

c. Rédigez un court rapport : vulnérabilités, CWE, gravité (U10), correctifs proposés.

7. Résultats attendus

Un rapport identifiant au moins une faille, son CWE et une remédiation.

8. Explications

L'audit combine repérage statique (points chauds) et confirmation dynamique (sanitizers/fuzzing).

9. Questions

1. Quelles fonctions à risque ?
2. Quelles entrées déclenchent un rapport ?
3. Quels correctifs, et comment les valider ?

11. Vulnérabilité & remédiation

Selon le code fourni. Remédiations adaptées (copie bornée, validation d'entrée, vérification d'overflow).

12. Durée

1 h 30.

10. Corrigé : livrable D, Lab 5.3.

Binaire d-analyse et exemples concus pour le cours ; sorties verifiees.